

Technical Handover & Production Deployment Guide

Application: Influencer Partnership & Sales Management Platform **Repository:** `Harsxch/Harsxch.github.io` (branch: `claude/influencer-sales-platform-q6bot0`) **Audit date:** 2026-09-23 **Audited by:** direct inspection of the codebase (package files, schema, routes, auth, business logic) — not a description from memory. Anything not verifiable in code is explicitly marked **REQUIRES TECH TEAM CONFIRMATION**.

Read this if you are the engineer or team taking this application from "built by Claude Code" to "running in production." It documents what exists today, what's mock vs. real, what's missing, and exactly what you need to do/provide to go live.

1. Executive Summary

This is a web platform for running an influencer/creator partnership program for an online course business: managing influencers, their commercial (revenue-share/commission) agreements, tracking links, coupons, campaigns and courses, calculating what each influencer earns per sale, and paying them out — plus a self-serve dashboard where influencers can see their own sales and performance.

Who uses it: - **Internal staff** (Super Admin, Finance, Influencer Manager, Analyst) — manage influencers, courses, agreements, campaigns, coupons, record sales, approve earnings, run payouts. - **Influencers** — log in to see their own dashboard, sales, tracking links, coupons, earnings, payouts, and (newest feature) a UTM/coupon-filterable Sales Tracking report.

Current scope: A working, fully-implemented core product on top of **mock/manually-entered data** — there is no live payment gateway, LMS, or Metabase connection wired in yet (see §12). The financial calculation engine, RBAC, and data-isolation logic are real and production-grade in design; what's missing before go-live is almost entirely **integration** (real data sources, hosting, CI/CD, secrets) rather than application logic.

Development status: Feature-complete for Phase 1 as scoped. No automated test suite exists beyond one manual smoke script (§25). No deployment pipeline exists yet (§17–18).

Before production launch, the tech team must: 1. Stand up a real Postgres database and hosting environment (§17). 2. Decide on and build the real Sales Tracking data source (currently mock data, §12). 3. Wire up whatever order-ingestion path (API/webhook/CSV) matches the real course platform — only manual entry exists today (§13.2 note under Known Limitations). 4. Rotate/generate all production secrets (§10) — do not reuse anything found in this environment's `.env`. 5. Add monitoring, backups, and CI (§17, §26, §27) — none exist today.

2. Product Overview

Module	Status	Notes
Influencer management (CRUD, profile, manager assignment)	IMPLEMENTED	Admin-side pages + Server Actions
Influencer dashboard (own performance)	IMPLEMENTED	Real DB-backed, not mock
Sales Tracking (UTM/coupon self-serve filtering)	PLACEHOLDER / MOCK	Fully built UI + API contract; data is synthetically generated, not from a real DB table or Metabase (§12)
Tracking links (admin-issued + influencer self-serve creation)	IMPLEMENTED	Influencer can create/deactivate their own links
Coupons	IMPLEMENTED	Admin-issued, linked to influencers/campaigns/courses
Courses	IMPLEMENTED	Admin CRUD, influencer access grants
Campaigns	IMPLEMENTED	Groups courses/influencers/coupons/tracking links
Commercial agreements & versioning	IMPLEMENTED	Append-only, versioned, scoped to course/campaign
Commission/revenue-share calculation	IMPLEMENTED	Pure <code>decimal.js</code> -based engine, 4 model types
Earnings ledger	IMPLEMENTED	Immutable per-transaction snapshots
Payouts	IMPLEMENTED	Bundles approved earnings; no real payment-provider integration (money movement itself is manual/external)
Refunds / reversals	IMPLEMENTED	Proportional reversal against original earning, supports partial refunds
Goals	IMPLEMENTED	Per-influencer target tracking
Notifications	IMPLEMENTED	In-app only
Marketing assets	PARTIALLY IMPLEMENTED	Data model + admin listing exist; no file upload — asset URLs are pasted in, not uploaded
Audit log	IMPLEMENTED	Append-only by construction, no update/delete path in the app
CSV exports (sales/earnings/payouts/influencers)	IMPLEMENTED	Synchronous, capped at 10,000 rows — flagged as a scale limit in existing docs
Order ingestion via real payment/LMS webhook or API	NOT IMPLEMENTED	Only manual entry by admin staff exists; schema has <code>OrderSource.API / WEBHOOK / CSV_IMPORT</code> values reserved for later
Email/WhatsApp notification delivery	NOT IMPLEMENTED	In-app notifications only
Payment gateway integration	NOT IMPLEMENTED	No Stripe/Razorpay/etc. anywhere in the code
Metabase integration	NOT IMPLEMENTED	See §12
File upload (images/documents)	NOT IMPLEMENTED	No upload endpoint exists
Automated tests / CI	NOT IMPLEMENTED	One manual Playwright smoke script only

3. User Roles & Access

Roles (`Role`enum`): `SUPER_ADMIN`, `FINANCE`, `INFLUENCER_MANAGER`, `ANALYST`, `INFLUENCER`.

Role	Can access	Can create/edit	Cannot access
SUPER_ADMIN	Everything	Everything, including staff users	—
FINANCE	Influencers/courses/campaigns/coupons (read), agreements/orders/refunds/transactions/payouts (read+write), exports	Agreements, orders, refunds, payouts	Staff user management, marketing assets
INFLUENCER_MANAGER	Influencers, courses, campaigns, tracking links, coupons, assets, goals (read+write)	Same list	Agreements (read-only), payouts, refunds, staff users
ANALYST	Read-only across almost everything	Nothing	All write actions, staff users, audit-log write
INFLUENCER	Only their own data: own profile, own assigned courses, own tracking links (read+write, self-serve), own coupons, own orders (PII-stripped), own ledger, own payouts, own goals, own Sales Tracking	Own tracking links only	Any other influencer's data, all admin resources, staff users, audit logs

Influencer data isolation — how it's enforced (this is the critical security property of the app):

There are **two independent, server-side layers**, both must pass on every request — the frontend never enforces this alone:

- Resource-level RBAC** — `src/lib/auth/rbac.ts` . `assertCan(role, resource, action)` throws a `ForbiddenError` (→ HTTP 403) if the role lacks permission for that resource/action at all.
- Row-level scoping** — `src/lib/auth/session.ts` : `- resolveInfluencerScope(sessionUser, requestedInfluencerId)` — if the caller's role is `INFLUENCER` , this **always** returns the session's own `influencerId` and silently ignores whatever id was requested. `- assertOwnInfluencerScope(sessionUser, requestedInfluencerId)` — throws `ForbiddenError("Cannot access another influencer's data")` if an `INFLUENCER` caller's session id doesn't match the requested id.

The **Sales Tracking** feature (the newest, and the one this handover's manager-facing sibling document focuses on) goes one step further: `src/lib/sales-tracking/service.ts` derives the influencer's identity **exclusively** from the server session and does not accept an influencer/user id as an input parameter **at all** — there is no field for a client to tamper with in the first place. This was a specific, explicit security requirement during development and was verified: a logged-in influencer hitting `/api/influencer/sales-tracking` cannot pass any parameter to see another influencer's data; a non-INFLUENCER role gets a 403.

Status: this enforcement is server-side, not just frontend-hidden. No security issue was found in this area during the audit. The one general caveat: `assertCan / assertOwnInfluencerScope` usage was verified directly in the sales-tracking, dashboard, and ledger data-access functions; it was **not independently re-verified line-by-line in every single `actions.ts` file** across the whole admin/influencer tree in this pass — recommend the tech team grep for any `src/lib/data/*.ts` or `actions.ts` function that queries by an id **without** first calling `resolveInfluencerScope / assertOwnInfluencerScope` as part of their own pre-launch security pass (§16).

4. Application Architecture

Diagram (Mermaid source — renders visually on GitHub/most Markdown viewers):

```
flowchart TD
    U["U[User: Admin / Influencer]"] --> FE["FE[Next.js App Router - Server + Client Components]"]
    FE --> MW["MW[Edge Middleware - src/middleware.ts]"]
    MW --> AUTH["AUTH[Auth.js v5 - JWT session]"]
    AUTH --> RSC["RSC[Server Components / Server Actions]"]
    RSC --> RBAC["RBAC[RBAC layer - rbac.ts]"]
    RBAC --> SCOPE["SCOPE[Row-level scoping - session.ts]"]
    SCOPE --> LIB["LIB[Business logic - src/lib]"]
    LIB --> PRISMA["PRISMA[Prisma 7 Client + adapter-pg]"]
    PRISMA --> PG["PG[(PostgreSQL 16)]"]

    FE -. "read-only API routes" .-> API["API[src/app/api/**]"]
    API --> RBAC

    subgraph "Sales Tracking specifically"
        ST["ST[Influencer Sales Tracking page]"] --> STAPI["STAPI[/api/influencer/sales-tracking/]"]
        STAPI --> STSVC["STsvc[sales-tracking/service.ts]"]
    end

    STSVC --> STADAPTER["STADAPTER[SalesTrackingAdapter interface]"]
    STADAPTER --> MOCK["MOCK[MockSalesTrackingAdapter]"]
    CURRENT["CURRENT: synthetic in-memory data"]
    STADAPTER -. "NOT YET BUILT" .-> METABASE["METABASE[(Metabase / real data source)]"]

    end
```

- Frontend & Backend:** same Next.js 16 App Router app — Server Components render pages, Server Actions handle almost all writes, a small set of route handlers (`src/app/api/**`) handle reads that need to be called from client-side `fetch` or return CSV.
- Database:** single PostgreSQL 16 instance, accessed only through Prisma.
- External services:** none currently (§9, §12).
- Storage:** none — no file upload capability exists.
- Background processes:** none — no cron/queue/worker of any kind.

5. Technology Stack

Technology	Version	Purpose	Notes
Next.js	16.3.5	Full-stack framework (App Router, Server Components/Actions, Turbopack)	
React / React DOM	19.2.8	UI runtime	

Technology	Version	Purpose	Notes
TypeScript	5.9.3	Language	
PostgreSQL	16	Primary database	Confirmed via local dev cluster (<code>pg_lsclusters</code>); production version REQUIRES TECH TEAM CONFIRMATION
Prisma	7.10.0 (<code>prisma</code> + <code>@prisma/client</code>)	ORM / schema / migrations	Uses the newer <code>prisma-client</code> generator with a custom output path <code>src/generated/prisma</code> (not the legacy <code>prisma-client-js</code>)
@prisma/adapters-pg	^7.10.0	Driver adapter required by Prisma 7's new client engine	Wraps <code>pg</code>
pg	^8.23.0	Postgres driver	
next-auth (Auth.js)	5.0.0-beta.32	Authentication, JWT sessions	Beta version — see §16
bcryptjs	^3.0.3	Password hashing	
Tailwind CSS	^4 (resolved 4.3.3)	Styling	
decimal.js	^10.6.0	Arbitrary-precision decimal math for all money calculations	Never floating point
zod	^4.6.5	Runtime input validation (Server Action / order schemas)	
lucide-react	^1.47.0	Icon set	
recharts	^3.10.1	Charts (dashboard revenue/earnings graphs)	
Playwright	^1.63.0 (dev dependency)	Browser automation	Used only for one ad hoc smoke script, not a configured test runner (§25)
tsx	^4.23.15 (dev)	Run TypeScript scripts directly (seed script)	
ESLint	^9 + eslint-config-next 16.3.5 (dev)	Linting	
npm	—	Package manager	<code>package-lock.json</code> present; no <code>pnpm</code> / <code>yarn</code> lockfile

No file storage/CDN library, no payment SDK, no email/SMS SDK, no error-tracking SDK (e.g. Sentry), and no logging library are present in the dependency tree.

6. Project Structure

Everything lives under `src/` (no top-level `/app`). Path alias `@/*` → `./src/*` .

```
/
├── .env.example           # committed, placeholders only
├── README.md             # quickstart + stack summary
├── docs/
│   ├── ARCHITECTURE.md   # design rationale, referenced by section number to an external spec
│   ├── TECHNICAL_HANOVER.md # this document
│   └── screenshots/      # screenshots referenced by this document
├── prisma.config.ts       # Prisma 7 CLI config (schema/migrations paths, DATABASE_URL)
├── prisma/
│   ├── schema.prisma     # full data model (§11)
│   ├── seed.ts           # dev/demo data generator – npm run db:seed
│   └── migrations/       # 2 migrations to date
├── scripts/
│   └── e2e-smoke.mjs      # manual Playwright smoke script (§25)
├── legacy-portfolio/      # UNRELATED – leftover static portfolio site from this repo's origin as a personal GitHub Pages site. Confirm with the team
└── whether it can be deleted.
    └── src/
        ├── middleware.ts  # coarse role-based route gate
        ├── generated/prisma/ # Prisma Client output (generated; not hand-edited)
        ├── app/
        │   ├── login/     # login page + Server Action
        │   ├── api/       # the ONLY 5 route handlers in the app (§14)
        │   │   ├── auth/[...nextauth]/route.ts
        │   │   ├── exports/{earnings,influencers,payouts,sales}/route.ts
        │   │   └── influencer/sales-tracking/route.ts
        │   ├── r/[code]/route.ts # public tracking-link redirect (not under /api)
        │   ├── admin/         # staff-facing pages + colocated Server Actions
        │   └── influencer/    # influencer-facing pages + colocated Server Actions
        ├── components/
        │   ├── charts/       # Recharts wrappers
        │   ├── filters/      # shared sales filter bar
        │   ├── layout/       # app-shell (sidebar/nav), sign-out button
        │   └── ui/           # badge, button, card, table, stat-card, etc.
        ├── lib/             # ALL business logic – the only place that talks to Prisma
        │   ├── auth/         # auth.config.ts, auth.ts, auth-edge.ts, rbac.ts, session.ts
        │   ├── financial/    # calculation engine, agreements, payouts, reversal
        │   ├── attribution/  # coupon/tracking-link attribution strategies
        │   ├── orders/      # order + refund ingestion pipeline (manual-entry only)
        │   ├── sales-tracking/ # the Sales Tracking feature (§12–13)
        │   ├── data/        # per-domain read queries (each enforces RBAC + scoping)
        └── coupons/, audit/, notifications/, api/, csv.ts, format.ts, date-range.ts, prisma.ts
```

A new engineer mainly needs: `src/lib/auth/` (how identity/permissions work), `src/lib/financial/` (how money is calculated), `src/lib/sales-tracking/` (the feature needing real data), `prisma/schema.prisma` (the data model), and the relevant `src/app/**` folder for whatever page they're changing.

7. Local Development Setup

Exact steps, taken from `package.json` and `README.md` (no invented commands):

```
# 1. Prerequisites
#   - Node.js (v22.x confirmed working in this environment; exact minimum REQUIRES TECH TEAM CONFIRMATION — no `engines` field is set)
#   - PostgreSQL 16 reachable locally
#   - npm

# 2. Install dependencies
npm install

# 3. Configure environment
cp .env.example .env
#   then fill in real values (see §10)

# 4. Apply database schema
npx prisma migrate deploy

# 5. (Optional) seed demo/dev data
npm run db:seed

# 6. Run the dev server
npm run dev # http://localhost:3000

# 7. Build for production
npm run build
npm start

# 8. Lint
npm run lint

# 9. Manual smoke test (requires steps 4–6 already done)
npm run e2e:smoke
```

There is **no `npm test` script** and **no automated unit/integration test command** (§25).

Seeded demo logins (from `prisma/seed.ts`, password `Passw0rd!` for all):

Email	Role
admin@platform.dev	SUPER_ADMIN
finance@platform.dev	FINANCE
manager@platform.dev	INFLUENCER_MANAGER
analyst@platform.dev	ANALYST
rahul@creator.dev / priya@creator.dev / amina@creator.dev	INFLUENCER

8. Environment Variables

Variable	Purpose	Required?	Example/Format	Used in	Production Notes
DATABASE_URL	Postgres connection string for Prisma	Yes	postgresql://user:YOUR_SECRET_HERE@host:5432/dbname?schema=public	src/lib/prisma.ts , prisma/seed.ts , prisma.config.ts	Must point at the production Postgres instance; use a pooled connection string if the host requires one (e.g. pgbouncer) — REQUIRES TECH TEAM CONFIRMATION on the chosen host's pooling requirements
AUTH_SECRET	JWT signing secret for Auth.js sessions	Yes	YOUR_SECRET_HERE (generate via openssl rand -base64 32)	Read implicitly by NextAuth v5 internals	Must be a fresh, random value in production — never reuse a dev value
NEXTAUTH_URL	Canonical app URL, used by Auth.js for callback construction	Yes	https://your-production-domain.com	Read implicitly by NextAuth v5 internals	Must exactly match the production domain (including protocol)
NODE_ENV	Standard Node environment flag	Set automatically by next build / next start	production	src/lib/prisma.ts (guards the dev-only hot-reload PrismaClient caching pattern)	No action needed — Next.js sets this

Variable	Purpose	Required?	Example/Format	Used in	Production Notes
BASE_URL	Target URL for the manual smoke script only	No (defaults to <code>http://localhost:3000</code>)	<code>https://staging.your-domain.com</code>	<code>scripts/e2e-smoke.mjs</code>	Not needed at runtime, only if the tech team runs the smoke script against a deployed environment
PLAYWRIGHT_EXECUTABLE_PATH	Optional Chromium path override for the smoke script	No	path to a Chromium binary	<code>scripts/e2e-smoke.mjs</code>	Only relevant in sandboxed CI environments where Playwright can't download its own browser

No frontend-exposed (`NEXT_PUBLIC_*`) variables exist. No Metabase, payment, email/SMS, or storage-related environment variables exist because those integrations are not implemented (\$2, \$12).

Security note found during this audit: a real `.env` file with live-looking `DATABASE_URL` / `AUTH_SECRET` / `NEXTAUTH_URL` values exists on disk in the development environment this app was built in. It is correctly excluded via `.gitignore` and was **not** committed to the repository (verified with `git ls-files` / `git check-ignore`). Treat any values in that file as compromised for a shared/dev database — generate fresh secrets for every new environment (dev, staging, production) rather than copying it forward.

9. Database / Data Architecture

Type: PostgreSQL 16, accessed only through Prisma 7 (`@prisma/adpater-pg` driver adapter). Schema at `prisma/schema.prisma` (2 migrations applied to date: `20260922134735_init` , `20260922155721_tracking_link_utm_term_and_coupon`).

All money fields are `Decimal(12,2)` (Postgres `NUMERIC`) — never floating point — matching the `decimal.js`-based calculation engine.

Core relationship chain

Diagram (Mermaid source — renders visually on GitHub/most Markdown viewers):

```
graph LR
    User -->|1:1| Influencer
    Influencer -->|has many, versioned| CommercialAgreement
    CommercialAgreement -->|has many, immutable| CommercialAgreementVersion
    Influencer --> TrackingLink
    Influencer --> Coupon
    Campaign --> TrackingLink
    Campaign --> Coupon
    Course --> TrackingLink
    Course --> Coupon
    TrackingLink -- attribution --> OrderAttribution
    Coupon -- attribution --> OrderAttribution
    Order -->|1:1| OrderAttribution
    Order --> FinancialTransaction
    CommercialAgreementVersion -- snapshot --> FinancialTransaction
    FinancialTransaction --> Payout
    Order --> Refund
    Refund -- creates REVERSAL --> FinancialTransaction
```

Key entities (selected — full model list is in `prisma/schema.prisma`)

Entity	Key fields	Notable constraints
User	email, passwordHash, role, isActive	<code>email</code> unique; <code>@index([role])</code>
Influencer	userId (1:1), managerId → User, status	<code>email</code> unique; indexed on status, managerId
CommercialAgreement / CommercialAgreementVersion	modelType, shares/commission/fixedAmount, effectiveFrom/Until, status	Versions are immutable and append-only; <code>@unique([agreementId, version])</code>
TrackingLink	code (unique), influencerId, courseId, utmSource/Medium/Campaign/Content/Term, couponId	Indexed on influencerId, courseId, status
Coupon	code (unique), influencerId, discountType/Value, agreementVersionId (override), usageLimit/currentUsage	Indexed on influencerId, status
Customer	email (unique), name, phone	Deliberately minimal — "influencers must never see customer PII" (schema comment, line ~497)
Order	orderNumber (unique), customerId, courseId, couponId, originalPrice/discountAmount/finalAmount	Indexed on courseId, status, placedAt, couponId
OrderAttribution	orderId (1:1, unique), influencerId, trackingLinkId, couponId, source	Indexed on influencerId, trackingLinkId, couponId
FinancialTransaction	type, status, immutable snapshot fields of the agreement terms used, eligibleRevenue/influencerAmount/companyAmount	Indexed on <code>[influencerId, status]</code> , orderId, payoutId; self-relation for REVERSAL – EARNING linkage
Payout	influencerId, amount, status, reference, method	Indexed on <code>[influencerId, status]</code>
AuditLog	action, entityType/Id, previousValue/newValue (JSON)	Append-only by application-layer convention — no update/delete path exists in the code

Full enum list: `Role` , `InfluencerStatus` , `AgreementModelType` , `EligibleRevenueBasis` , `AgreementStatus` , `CourseStatus` , `CampaignStatus` , `TrackingLinkStatus` , `CouponDiscountType` , `CouponStatus` , `OrderStatus` , `OrderSource` , `AttributionSource` , `TransactionType` , `TransactionStatus` , `PayoutStatus` , `GoalMetric` , `AssetType` ,

NotificationType .

10. Metabase / Data Source Integration

Metabase integration is not currently implemented in the application. A repo-wide case-insensitive search for "metabase" returns matches only inside code comments in `src/lib/sales-tracking/` describing it as a **future** integration point — there is no Metabase SDK dependency, no API base URL, no connection string, and no HTTP call to Metabase anywhere in the code.

CURRENT IMPLEMENTATION (Sales Tracking data)

The Sales Tracking feature is built against a clean adapter interface specifically so a real data source can be dropped in later without touching the UI or API route:

```
// src/lib/sales-tracking/adapter.ts
export interface SalesTrackingAdapter {
  getFilterOptions(identity: InfluencerIdentity): Promise<SalesFilterOptions>;
  getSales(identity: InfluencerIdentity, filters: SalesFilters): Promise<InfluencerSalesResponse>;
}

// src/lib/sales-tracking/index.ts <-- THE SINGLE SWAP POINT
export const salesTrackingAdapter: SalesTrackingAdapter = new MockSalesTrackingAdapter();
```

Today, `MockSalesTrackingAdapter` (`src/lib/sales-tracking/mock-adapter.ts`) generates **deterministic synthetic data in memory** (`mock-data.ts` — seeded per-influencer-id PRNG, 35–54 fake sales rows from fixed course/UTM/coupon vocab lists, cached per server process). There is no database table backing it and no real order data is read.

EXPECTED PRODUCTION DATA INTEGRATION

To go live, the tech team implements a new class satisfying the same `SalesTrackingAdapter` interface (e.g. `MetabaseSalesTrackingAdapter`) and changes the one export in `index.ts`. Nothing else in the app needs to change.

API endpoint the frontend calls: `GET /api/influencer/sales-tracking`

Query parameters (all optional; absence = no filter applied):

Param	Type	Example
<code>startDate</code>	ISO date string	<code>2026-06-01</code>
<code>endDate</code>	ISO date string	<code>2026-09-30</code>
<code>utmSource</code>	string	<code>youtube</code>
<code>utmMedium</code>	string	<code>influencer</code>
<code>utmCampaign</code>	string	<code>dedicated</code>
<code>utmContent</code>	string	<code>sf</code>
<code>utmTerm</code>	string	<code>HarshPriyam</code>
<code>couponCode</code>	string	<code>HPDSPAPP</code>
<code>courseName</code>	string	<code>Python + DSA Mastery</code>

Note: identity (which influencer) is **never** a parameter — it comes only from the server-side session (§3).

Response shape (`InfluencerSalesResponse`):

```
{
  "summary": {
    "totalSales": 51,
    "totalRevenue": 597800,
    "coursesSold": 3
  },
  "courseBreakdown": [
    { "courseName": "AI Mastery", "sales": 16, "revenue": 232500 }
  ],
  "utmBreakdown": [
    { "utmContent": "sf", "sales": 13, "revenue": 155050 }
  ],
  "sales": [
    {
      "orderId": "ORD-MUDJS9YP-WC28",
      "date": "2026-09-18",
      "courseName": "Python + DSA Mastery",
      "utmSource": "storefront",
      "utmMedium": "V_Upskill_Academy",
      "utmCampaign": "Dedicated",
      "utmContent": "sf",
      "utmTerm": "HarshPriyam",
      "couponCode": "HPDSPAPP",
      "userId": "usr_9f2ab1",
      "saleAmount": 10000,
      "saleStatus": "PAID"
    }
  ]
}
```

Required fields per sale row: `orderId`, `date`, `courseName`, `userId`, `saleAmount`, `saleStatus`. Optional/nullable: `utmSource`, `utmMedium`, `utmCampaign`, `utmContent`, `utmTerm`, `couponCode`. `saleStatus` is one of `"PAID"` | `"PENDING"` | `"REFUNDED"` | `"CANCELLED"`.

11. Sales Tracking Feature

Access: Influencer logs in → "Sales Tracking" in the left nav (directly under "My Sales").

Flow:

Diagram (Mermaid source — renders visually on GitHub/most Markdown viewers):

```
graph LR
    A[Logged-in Influencer] --> B[Sales Tracking page loads  
empty state - no fetch yet]
    B --> C[Pick date range + optional UTM/coupon/course filters]
    C --> D[Click Apply Filters]
    D --> E[GET /api/influencer/sales-tracking]
    E --> F[service.ts resolves identity from session ONLY]
    F --> G[salesTrackingAdapter.getSales]
    G --> H[Mock adapter today / Metabase adapter later]
    H --> I[Summary cards + course breakdown + UTM breakdown + detail table]
```

- **Filters:** Start Date & End Date (both required before any fetch happens), UTM Source, UTM Medium, UTM Campaign, UTM Content, UTM Term, Coupon Code, Course Name (all optional, populated dynamically per-influencer from `getFilterOptions`).
- **States:** idle (default — "select a date range," no data loaded), loading (spinner), error ("unable to load, retry" button), loaded-empty ("no sales found for the selected filters"), loaded-with-data.
- **Data shown on success:** 3 summary cards (Total Sales, Total Revenue, Courses Sold), a course-wise breakdown table, a UTM-content breakdown table, and a full sales detail table (date, course, all UTM fields, coupon, opaque User ID, order ID, amount, status).
- **Reset:** "Clear Filters" resets all filters and returns to the idle state.
- **Pagination:** **NOT IMPLEMENTED** — the detail table renders all matching rows at once. **Export:** **NOT IMPLEMENTED** for this specific feature (the separate `/api/exports/sales` admin-only CSV export is unrelated and not influencer-facing).
- **Screenshots** are in §21.

12. API Documentation

All 5 route handlers in the app, verbatim from `src/app/api/**/route.ts`. Everything else (agreements, campaigns, coupons, courses, orders, payouts, tracking-links CRUD) is implemented as **Server Actions**, not REST endpoints — there is no separate JSON API for those.

GET/POST `/api/auth/{...nextauth}`

NextAuth's own internal machinery (signin/callback/session/csrf). Not hand-written; no custom logic to document.

GET `/api/influencer/sales-tracking`

- **Auth:** session required; role must be `INFLUENCER` with a resolved `influencerId`, or 403.
- **Authorization:** identity comes only from the session — no id parameter accepted.
- **Query params / response:** see §12 (Metabase section) above.
- **Errors:** 401 not authenticated, 403 wrong role, 500 unexpected error (all via the shared `apiHandler` wrapper, §16).

GET `/api/exports/earnings`

- **Auth:** session + `assertCan(role, "exports", "read")` — `SUPER_ADMIN/FINANCE/INFLUENCER_MANAGER` only.
- **Query params:** `status` (transaction status filter — passed through without runtime validation against the enum).
- **Response:** CSV file (`earnings.csv`).
- **Note:** no row-level influencer scoping is applied — any permitted role receives ALL influencers' earnings.

GET `/api/exports/influencers`

- **Auth:** same `assertCan` pattern.
- **Query params:** none.
- **Response:** CSV (`influencers.csv`), capped at 10,000 rows.

GET `/api/exports/payouts`

- **Auth:** same pattern. **Query params:** none. **Response:** CSV (`payouts.csv`).

GET `/api/exports/sales`

- **Auth:** same pattern.
- **Query params:** `status`, `courseId`, `campaignId`, `influencerId`, `couponCode`, `utmSource`, `utmMedium`, `utmCampaign`, `utmContent`, `utmTerm`, `from`, `to` — all forwarded to `listOrders()`.
- **Response:** CSV (`sales.csv`).

GET `/r/{code}` (public, not under `/api`)

- **Auth:** none — intentionally public.
- **Purpose:** tracking-link redirect. Looks up the `TrackingLink` by `code`; if missing/inactive, redirects to `/`; otherwise increments the click counter and 302-redirects to the course URL with the link's full UTM parameters and coupon code appended.

13. Authentication

- **Login flow:** single email+password Credentials provider (Auth.js v5 / NextAuth). `authorize()` (`src/lib/auth/auth.ts`) looks up the user by lowercased/trimmed email, rejects inactive users, compares the password with `bcrypt.compare`, updates `lastLoginAt`, and returns `{ id, email, name, role, influencerId }`.
- **Session management:** JWT strategy (no database session table). Role and `influencerId` are copied onto the JWT at sign-in and exposed on `session.user` via the session callback.
- **Token management:** handled entirely by Auth.js's own JWT encode/decode using `AUTH_SECRET`; no custom token logic.

- **Password handling:** bcryptjs hash, never stored/logged in plaintext.
- **Identifying the logged-in influencer:** every server-side function calls `requireSession()` (`src/lib/auth/session.ts`) which is the sole source of truth for “who is calling”; `session.user.influencerId` is what ties a request to a specific `Influencer` row — never a client-supplied parameter.
- **Logout:** standard Auth.js sign-out (`SignOutButton` component).
- **Session expiration:** governed by Auth.js's default JWT session settings — no custom `maxAge` /rotation was found configured; **exact expiry duration REQUIRES TECH TEAM CONFIRMATION** if a specific policy is needed.
- **Password reset: NOT IMPLEMENTED** — no reset-flow pages, tokens, or email sending exist.
- **Edge/Node split:** Next.js Edge Middleware can't load the generated Prisma client, so auth is split into `auth.config.ts` (edge-safe, stub `authorize`), `auth.ts` (full config with the real `Prisma/bcrypt authorize` , used everywhere except middleware), and `auth-edge.ts` (edge-safe `auth()` used only by `middleware.ts`).

14. Security Review

Area	Finding
Exposed secrets / hardcoded credentials	None found in source. A real <code>.env</code> exists on disk in this dev environment (correctly git-ignored) — rotate before any shared use (§8, §10).
Unsafe API endpoints	All 5 route handlers are wrapped in <code>apiHandler</code> with explicit auth checks; none found unauthenticated except the intentionally-public <code>/r/[code]</code> redirect.
Missing authorization	Not found in the paths independently verified (sales-tracking, dashboard, ledger). Not exhaustively re-verified across every Server Action — recommend a full grep pass before launch (§3).
Client-side-only access control	None found — the influencer data-isolation property is enforced server-side in two independent layers (§3), not just hidden in the UI.
SQL injection	Low risk — all queries go through Prisma's parameterized query builder; no raw SQL string concatenation was found.
XSS	Low risk — standard React/JSX escaping throughout; no <code>dangerouslySetInnerHTML</code> usage found.
CSRF	Handled by Auth.js's built-in CSRF protection for its own endpoints; Server Actions get Next.js's built-in Origin-header protection. Not independently penetration-tested.
Insecure direct object references (IDOR)	Sales Tracking specifically has no id parameter to tamper with at all (identity is session-derived). Admin export routes (<code>/api/exports/sales</code> etc.) accept an <code>influencerId</code> filter, but access to those routes is already gated to admin-tier roles by RBAC — not a per-influencer IDOR risk since influencers can't reach them.
Excessive API responses	Admin-only CSV exports return full unfiltered rows (capped at 10,000) to any role with <code>exports</code> permission — by design, those roles are meant to see all data.
Unsafe file uploads	N/A — no upload functionality exists.
Debug endpoints / dev credentials	<code>prisma/seed.ts</code> creates known dev credentials (<code>Passw0rd!</code> for all seeded accounts) — must not be run against production , or if it is for initial setup, all seeded passwords must be rotated immediately.
CORS	No custom CORS configuration found in <code>next.config.ts</code> (default/empty config) — Next.js's same-origin defaults apply.
Production configuration issues	No environment-specific config split was found beyond <code>NODE_ENV</code> ; <code>next-auth</code> is on a beta version (<code>5.0.0-beta.32</code>) — confirm this is an acceptable production dependency before launch.

CRITICAL BEFORE PRODUCTION

- [] Rotate every secret found in this development environment's `.env` — do not carry any of them into staging/production.
- [] Run a full grep for every `src/lib/data/*.ts` function and `admin/influencer actions.ts` file to confirm `assertOwnInfluencerScope / resolveInfluencerScope` is called before any influencer-scoped query (only a sample was independently re-verified in this audit).
- [] Decide whether to pin `next-auth` off its current beta version before production, or explicitly accept the beta as final for launch.
- [] Ensure `npm run db:seed` (which creates known dev passwords) is never run against the production database, or that all seeded accounts have their passwords rotated immediately if it is used for initial data setup.

15. Deployment Architecture

CURRENT CONFIGURATION

- `next.config.ts` is empty/default — no custom rewrites, headers, image domains, or output mode set.
- No `vercel.json`, no `Dockerfile`, no `docker-compose.yml`, no `.github/workflows/` — **no deployment or CI configuration of any kind exists in the repo.**
- README explicitly states the app **cannot** run on GitHub Pages (needs a database, server-side auth, and the financial calculation engine — none of which a static host can provide) and suggests a Node-capable host (Vercel/Railway/Render/Fly.io, unconfirmed which one this org will use) with a managed Postgres provider (Neon/Supabase/Railway/RDS, likewise unconfirmed).
- Documented (README) manual deploy steps: `npm install` → configure `.env` → `npx prisma migrate deploy` → (`npm run db:seed` optionally, dev only) → `npm run build` → `npm start` .

RECOMMENDED PRODUCTION CONFIGURATION — REQUIRES TECH TEAM CONFIRMATION

The following all depend on decisions only the tech team can make; nothing below is inferred from the code: - **Frontend/Backend hosting:** any Node.js-capable host (this is a single full-stack Next.js app, not a static site) — **REQUIRES TECH TEAM CONFIRMATION.** - **Database hosting:** managed Postgres 16-compatible service — **REQUIRES TECH TEAM CONFIRMATION.** - **Domain / SSL:** not configured anywhere in the repo — **REQUIRES TECH TEAM CONFIRMATION.** - **Migrations on deploy:** run `npx prisma migrate deploy` as a release step before `npm start` ; no automation for this exists yet. - **Auth callback URLs:** `NEXTAUTH_URL` must be set to the real production domain. - **CORS / webhooks / storage / CDN:** not applicable today since none of those integrations exist yet (§2, §9, §10).

16. Production Deployment Checklist

```
[ ] Production repository/branch strategy confirmed
[ ] Production hosting environment chosen and created (frontend + Node runtime)
[ ] Production PostgreSQL 16+ database provisioned
[ ] Environment variables configured (DATABASE_URL, AUTH_SECRET, NEXTAUTH_URL) – see §8
[ ] All secrets rotated from dev values (never reused from this environment)
[ ] Database migrations executed (`npx prisma migrate deploy`)
```

```
[ ] Prisma Client generated as part of the build/install step
[ ] Real Sales Tracking data adapter implemented and swapped in (§10) – or explicitly deferred with stakeholders' sign-off that mock data ships at launch
[ ] Authentication domain/callback URL configured for production
[ ] Domain configured and DNS pointed
[ ] SSL/TLS active
[ ] Application deployed and reachable
[ ] Seeded demo/dev accounts removed or passwords rotated if seed script was used for initial setup
[ ] Admin access tested end-to-end (login, all admin nav pages)
[ ] Influencer access tested end-to-end (login, dashboard, Sales Tracking, My Links)
[ ] Influencer data-isolation re-tested against production data (cannot access another influencer's data via any parameter)
[ ] Sales Tracking tested against whatever data source is live at launch
[ ] UTM filtering, coupon filtering, course filtering all tested individually and combined
[ ] Empty-result and error states tested
[ ] Mobile responsiveness spot-checked (already verified at 1440px/390px during development; tablet width was explicitly descoped)
[ ] A full security pass completed (see §14 Critical list)
[ ] Logging/error visibility configured beyond default stdout capture
[ ] Monitoring configured (§26 – nothing exists today)
[ ] Backup strategy confirmed for the production database (§27 – nothing exists today)
[ ] Rollback strategy confirmed for both app deploys and DB migrations (§27)
[ ] CI pipeline set up (none exists today) running at minimum `npm run lint` and `npm run build`
```

17. Information Required From Tech Team

Only items the application genuinely needs before it can go live — not a generic wishlist:

- Production `DATABASE_URL` (host, credentials, pooling requirements).
- Production `AUTH_SECRET` and confirmed production domain for `NEXTAUTH_URL` .
- Decision + implementation plan for the real Sales Tracking data source (Metabase connection details, or whatever the actual system of record is) — see §10 for the exact interface it must satisfy.
- Real order/sale data schema mapping so a real `SalesTrackingAdapter` (or the underlying order-ingestion pipeline) can be written — see §18 Data Mapping.
- Decision on which order-ingestion path (real payment/LMS API, webhook, or scheduled CSV import) will replace today's manual-entry-only flow, and access/credentials for that source.
- Hosting environment and deployment process the org wants to standardize on (nothing is currently assumed).
- Any organizational requirement for password reset, SSO, or MFA (none of these exist today — confirm whether they're needed for launch).

18. Data Mapping (Sales Tracking)

Frontend field	API/type field	Source field (real system)	Data type	Example
Start Date / End Date	<code>startDate / endDate</code>	REQUIRES TECH TEAM CONFIRMATION (likely order placement date)	ISO date string	2026-06-01
UTM Source	<code>utmSource</code>	REQUIRES TECH TEAM CONFIRMATION (likely tracking-link/campaign UTM field in the real ad/analytics system)	string null	storefront
UTM Medium	<code>utmMedium</code>	REQUIRES TECH TEAM CONFIRMATION	string null	V_Upskill_Academy
UTM Campaign	<code>utmCampaign</code>	REQUIRES TECH TEAM CONFIRMATION	string null	Dedicated
UTM Content	<code>utmContent</code>	REQUIRES TECH TEAM CONFIRMATION	string null	sf
UTM Term	<code>utmTerm</code>	REQUIRES TECH TEAM CONFIRMATION	string null	HarshPriyam
Coupon Code	<code>couponCode</code>	REQUIRES TECH TEAM CONFIRMATION (likely the coupon/discount-code field on the order record)	string null	HPDSPAPP
Course Name	<code>courseName</code>	REQUIRES TECH TEAM CONFIRMATION (course/product name or ID in the real catalog)	string	Python + DSA Mastery
User ID	<code>userId</code>	REQUIRES TECH TEAM CONFIRMATION — must be an existing opaque, non-PII internal identifier; never phone/email/name	string	usr_9f2ab1
Sale Amount	<code>saleAmount</code>	REQUIRES TECH TEAM CONFIRMATION (final/net amount charged, matching whatever revenue basis the business reports on)	number	10000
Sale Status	<code>saleStatus</code>	REQUIRES TECH TEAM CONFIRMATION — must map onto <code>"PAID" \ "PENDING" \ "REFUNDED" \ "CANCELLED"</code>	enum string	PAID

This app's internal `Order / OrderAttribution` schema (§9) already models UTM source/medium/campaign/content/term and coupon linkage for the **manual-entry** flow that exists today — if the real data source is a different system entirely (e.g. a separate storefront/LMS whose data lives in Metabase), the mapping above needs the tech team to fill in the right column/field names on their side.

19. Screenshots

All captured live from the running application (seeded demo data), desktop viewport (1440×900). Saved under `docs/screenshots/` .

Sales Tracking — default state (before selecting a date range)

Influencer Partner Platform

Dashboard

My Sales

Sales Tracking

My Courses

My Links

My Coupons

Campaigns

Earnings

Payouts

Goals

Notifications

Profile

Rahul Sharma

Influencer

Logout

Sales Tracking

Check the sales generated through your links and campaigns — no spreadsheets, no waiting on the team.

Start Date

mm/dd/yyyy

End Date

mm/dd/yyyy

UTM Source

All

UTM Medium

All

UTM Campaign

All

UTM Content

All

UTM Term

All

Coupon Code

All

Course Name

All

Apply Filters

Clear Filters

Select a date range to view your sales

Add UTM, coupon, or course filters if you want to narrow it down.

Sales Tracking — filters selected, before applying

Influencer Partner Platform

Dashboard

My Sales

Sales Tracking

My Courses

My Links

My Coupons

Campaigns

Earnings

Payouts

Goals

Notifications

Profile

Rahul Sharma

Influencer

Logout

Sales Tracking

Check the sales generated through your links and campaigns — no spreadsheets, no waiting on the team.

Start Date

06/01/2026

End Date

06/30/2026

UTM Source

All

UTM Medium

All

UTM Campaign

All

UTM Content

All

UTM Term

All

Coupon Code

All

Course Name

All

Apply Filters

Clear Filters

Select a date range to view your sales

Add UTM, coupon, or course filters if you want to narrow it down.

Influencer Partner Platform

Dashboard

My Sales

Sales Tracking

My Courses

My Links

My Coupons

Campaigns

Earnings

Payouts

Goals

Notifications

Profile

Sales Tracking

Check the sales generated through your links and campaigns — no spreadsheets, no waiting on the team.

Start Date

06/01/2026

End Date

09/30/2026

UTM Source

All

UTM Medium

All

UTM Campaign

All

UTM Content

All

UTM Term

All

Coupon Code

All

Course Name

All

Apply Filters

Clear Filters

TOTAL SALES

51

TOTAL REVENUE

₹5,97,800

COURSES SOLD

3

Course-wise breakdown

COURSE	SALES	REVENUE
AI Mastery	16	₹2,32,500
Data Science Bootcamp	16	₹1,84,800
Python + DSA Mastery	19	₹1,80,500

UTM content breakdown

Which specific video/post/link is generating sales

UTM CONTENT	SALES	REVENUE
sf	13	₹1,55,050
reel_01	10	₹1,22,550
video_03	10	₹1,13,450
video_01	9	₹1,06,000
video_02	9	₹1,00,750

Sales

51 orders matching your filters

DATE	COURSE	UTM SOURCE	UTM MEDIUM	UTM CAMPAIGN	UTM CONTENT	UTM TERM	COUPON	USER ID	ORDER ID
20 Sept 2026	AI Mastery	youtube	social	Dedicated	video_02	—	PRIYA15	v_0576362498647	0432892953a6a09a4a1fec32
19 Sept 2026	Python + DSA Mastery	youtube	social	FestiveSale	sf	HarshPriyam	RAHUL10	v_7397612521423	7cdf9f26c6c2c374a0057847
18 Sept 2026	Python + DSA Mastery	youtube	V_Upskill_Academy	Dedicated	reel_01	—	PRIYA15	v_8617835594919	cdbdcec4d8a953159a410133
17 Sept 2026	AI Mastery	instagram	social	Dedicated	sf	AminaKhan	—	v_7028913933153	ff81924a5fd4035dd70835ec
16 Sept 2026	AI Mastery	storefront	social	FestiveSale	video_03	—	SAVE500	v_0517820775522	7ed79175166576544d2d60a2
16 Sept 2026	Data Science Bootcamp	instagram	influencer	SeptemberLaunch	video_01	—	RAHUL10	v_1966534327351	8e58a4fd9f673d2b863c5f3
15 Sept 2026	Python + DSA Mastery	storefront	influencer	SeptemberLaunch	video_02	PriyaVerma	—	v_5582314698572	149d710d9bc120e8c776ec8c
04 Sept 2026	AI Mastery	storefront	influencer	Dedicated	reel_01	AminaKhan	PRIYA15	v_2301567790197	25e768501a784d7ba8a10cb5
02 Sept 2026	AI Mastery	instagram	social	Dedicated	sf	HarshPriyam	RAHUL10	v_0740668442271	80bc0514bd7db5b529c0a672
28 Aug 2026	Python + DSA Mastery	storefront	social	FestiveSale	reel_01	—	SAVE500	v_2704561947682	b54ea2b088e0a48a7271baed
26 Aug 2026	Data Science Bootcamp	storefront	influencer	Dedicated	reel_01	PriyaVerma	—	v_7468201024601	f9fd94fc7fcc6ab15b55f5df

25 Aug 2026	Data Science Bootcamp	storefront	influencer	SeptemberLaunch	video_01	—	SAVE500	v_1958561575411	e55a0832a6ab4f636c628c91
25 Aug 2026	Data Science Bootcamp	storefront	social	Dedicated	video_03	—	PRIYA15	v_1175007499256	86609566ffb9c9b387779cb7
23 Aug 2026	Data Science Bootcamp	instagram	social	FestiveSale	video_02	AminaKhan	PRIYA15	v_7531611124430	f76dfe330e2124056efe2df4
22 Aug 2026	Python + DSA Mastery	instagram	influencer	FestiveSale	reel_01	—	—	v_0967142086042	90109620cbb99fc109abe834
22 Aug 2026	Python + DSA Mastery	instagram	social	FestiveSale	video_01	—	PRIYA15	v_4840289813318	fc87210aee52d2e96ee0ce67
14 Aug 2026	Python + DSA Mastery	instagram	V_Upskill_Academy	FestiveSale	video_03	HarshPriyam	SAVE500	v_6398811311614	7579969b0e6b28e6ab33357e
12 Aug 2026	Data Science Bootcamp	instagram	influencer	SeptemberLaunch	sf	AminaKhan	PRIYA15	v_5959161995079	4b9f3d123e630ff3a8ff7a761
09 Aug 2026	Python + DSA Mastery	youtube	V_Upskill_Academy	Dedicated	sf	—	SAVE500	v_4237989483869	10c31d277298a9b0bcd11e85
08 Aug 2026	AI Mastery	youtube	influencer	Dedicated	sf	HarshPriyam	PRIYA15	v_9389655278437	16468eb465fd00110e9010ba
07 Aug 2026	Data Science Bootcamp	youtube	V_Upskill_Academy	Dedicated	sf	HarshPriyam	—	v_3531984807039	92e0ca73e71330e51e3a0afb
06 Aug 2026	Python + DSA Mastery	instagram	V_Upskill_Academy	Dedicated	video_01	—	—	v_7784048063015	fdd1d64f135699eeb12617d7
03 Aug 2026	Python + DSA Mastery	youtube	influencer	FestiveSale	video_03	HarshPriyam	PRIYA15	v_4245378322316	968e98185865230c49a5f433
02 Aug 2026	AI Mastery	instagram	social	Dedicated	sf	PriyaVerma	SAVE500	v_9546150752448	cdbdd3f68554e8e49be4d6bf
02 Aug 2026	Python + DSA Mastery	storefront	social	FestiveSale	video_01	—	SAVE500	v_9513581560227	60d156009e1b1482197d7114
31 Jul 2026	Data Science Bootcamp	youtube	influencer	Dedicated	video_03	—	SAVE500	v_2641739280583	33e90ee6dca975af181bb1b0
31 Jul 2026	Data Science Bootcamp	youtube	V_Upskill_Academy	FestiveSale	sf	—	—	v_1228739661293	3f414eec1112a39354eb64a9
28 Jul 2026	AI Mastery	storefront	social	SeptemberLaunch	video_01	AminaKhan	SAVE500	v_3578042804562	6094410660e822365972a80a
26 Jul 2026	Python + DSA Mastery	youtube	social	FestiveSale	sf	AminaKhan	—	v_0343583402194	49c1a61839287f2fe6f92853
26 Jul 2026	Data Science Bootcamp	youtube	V_Upskill_Academy	SeptemberLaunch	video_02	—	PRIYA15	v_5940909463671	02df872b3647a2e03a2a4492
25 Jul 2026	AI Mastery	storefront	V_Upskill_Academy	SeptemberLaunch	reel_01	AminaKhan	—	v_9580745582382	bbaf0fa9a7ecfe9c60178159
24 Jul 2026	Data Science Bootcamp	instagram	V_Upskill_Academy	SeptemberLaunch	reel_01	AminaKhan	PRIYA15	v_3785024857411	a65bd2217fa066798a6c650a
19 Jul 2026	Python + DSA Mastery	instagram	V_Upskill_Academy	Dedicated	video_03	AminaKhan	—	v_3117197898500	c250cd414800bae7cc927078
16 Jul 2026	Python + DSA Mastery	storefront	social	SeptemberLaunch	video_03	PriyaVerma	—	v_2939825583599	6a2c8d0af7283350472c4e76
15 Jul 2026	Python + DSA Mastery	youtube	influencer	FestiveSale	sf	PriyaVerma	—	v_5157800911708	37ba1680c7471d6ad87c8d7c
15 Jul 2026	Data Science Bootcamp	instagram	influencer	SeptemberLaunch	video_02	PriyaVerma	SAVE500	v_8055658210559	04c12408adedf1a2a34b6006
14	Python +								

Rahul Sharma Influencer [→ Sign out]	Jul 2026	DSA Mastery	storefront	social	SeptemberLaunch	sf	PriyaVerma	SAVE500	v_4653315519608	71c859e4d846bdd5938f5ba5
	14 Jul 2026	Data Science Bootcamp	youtube	influencer	SeptemberLaunch	video_03	—	SAVE500	v_8881697708756	30a6e33783d58d38e637613a
	14 Jul 2026	Data Science Bootcamp	youtube	influencer	FestiveSale	video_02	HarshPriyam	RAHUL10	v_7933217151357	4b9ef1b6f399d6eb25510ef4
	13 Jul 2026	AI Mastery	storefront	V_Upskill_Academy	FestiveSale	reel_01	—	PRIYA15	v_8469265583121	fa16b7221c764eeff2fa8f26
	12 Jul 2026	AI Mastery	instagram	social	Dedicated	video_01	AminaKhan	SAVE500	v_9696145555564	5c95c0075aeb2f74dcb173c6
	07 Jul 2026	Python + DSA Mastery	youtube	influencer	Dedicated	video_01	HarshPriyam	—	v_0348810952484	47d973b347389f51db1d9f8d
	05 Jul 2026	AI Mastery	storefront	influencer	SeptemberLaunch	reel_01	—	—	v_4976343730805	1cd29bc990ec0a18e959164d
	05 Jul 2026	Python + DSA Mastery	instagram	influencer	SeptemberLaunch	video_02	—	PRIYA15	v_368408066157	ed2a8496d1053771776c5172
	01 Jul 2026	AI Mastery	instagram	social	SeptemberLaunch	video_01	HarshPriyam	—	v_6127158960675	44f305c4267b029f84930463
	01 Jul 2026	AI Mastery	instagram	influencer	SeptemberLaunch	reel_01	HarshPriyam	SAVE500	v_3058656166421	43d54adfcdc133c8fd080ce2d
	30 Jun 2026	AI Mastery	youtube	social	Dedicated	video_03	—	SAVE500	v_4470360064173	242ffbfbf27a9d8afb57fdb97f
	30 Jun 2026	AI Mastery	instagram	influencer	SeptemberLaunch	sf	—	PRIYA15	v_8862775622384	c0733667fb70fc6c3e696193
	29 Jun 2026	Data Science Bootcamp	youtube	social	SeptemberLaunch	video_02	PriyaVerma	—	v_4974746460606	3358ba2a061e3b497e1dd26b
	28 Jun 2026	Python + DSA Mastery	storefront	social	Dedicated	video_02	HarshPriyam	SAVE500	v_7932993228251	676b1f36b5d8792c58ebb05c
	26 Jun 2026	Data Science Bootcamp	storefront	social	FestiveSale	video_03	—	RAHUL10	v_0710939252410	ddf9d9179525305e18bb8a58e

Influencer self-serve tracking links ("My Links")

Influencer Partner Platform

Dashboard

My Sales

Sales Tracking

My Courses

My Links

My Coupons

Campaigns

Earnings

Payouts

Goals

Notifications

Profile

My Links

Build your own link: pick a course, set your own UTM source/medium/campaign/content/term, and optionally attach one of your coupons.

Create a link

Course *

Select...

Platform *

Select...

Campaign

None

Attach one of my coupons

None

Content label (e.g. "Reel 3")

UTM PARAMETERS - SET YOUR OWN

UTM source

storefront

UTM medium

V_Upskill_Academy

UTM campaign

Dedicated

UTM content *

sf

UTM term

HarshPriyam

Create My Link

LINK		COURSE	PLATFORM	UTM SOURCE / MEDIUM	UTM CAMPAIGN / CONTENT / TERM	COUPON	CLICKS	SALES	REVENUE	EARNINGS	STATUS
/i/rahu1-sha1ma-data-science-bootcamp-sf2	Copy	Data Science Bootcamp	Instagram	storefront / V_Upskill_Academy	SelfServeTest / sf2 / RahulSelfServe	-	0	0	₹0	₹0	ACTIVE
/i/rahu1-python-dedicated-sf	Copy	Python + DSA Mastery	Other	storefront / V_Upskill_Academy	Dedicated / sf / HarshPriyam	HPDSPAPP	0	0	₹0	₹0	ACTIVE
/i/rahu1-python-yt-	Copy	Python + DSA	YouTube	youtube / influencer	september_launch / rahu1_video_1 /	-	0	2	₹30,000	₹15,000	ACTIVE

N

Rahul Sharma

Influencer

Influencer Partner Platform

Dashboard

Influencers

Courses

Campaigns

Tracking Links

Coupons

Sales

Commercial Agreements

Earnings

Payouts

Audit Logs

Dashboard

Influencer channel performance overview

TOTAL INFLUENCERS

3

3 active

TOTAL SALES

10

TOTAL REVENUE

₹1,13,500

INFLUENCER EARNINGS

₹27,575

COMPANY RETAINED

₹58,925

PENDING PAYOUTS

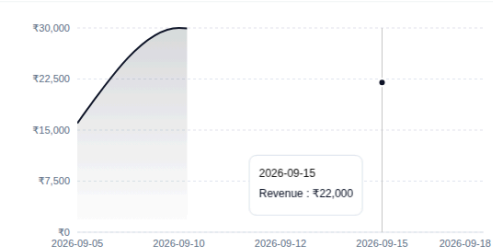
₹0

PAID PAYOUTS

₹0

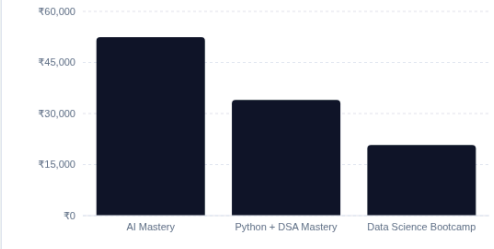
Revenue over time

Final amount collected per day



Date	Revenue
2026-09-05	₹15,000
2026-09-10	₹22,000
2026-09-15	₹22,000

Revenue by course



Course	Revenue
AI Mastery	₹45,000
Python + DSA Mastery	₹30,000
Data Science Bootcamp	₹15,000

Top influencers

By revenue in range

Influencer	Sales	Revenue	Earnings
Rahul Sharma	6	₹66,000	₹23,000

Recent orders

Order	Course	Influencer	Amount	Status
ORD-MUDJS9YP-WC28	Python + DSA Mastery	Rahul Sharma	₹10,000	SUCCESSFUL

Super Admin

Sign out

Influencer Partner Platform

Dashboard

Influencers

Courses

Campaigns

Tracking Links

Coupons

Sales

Commercial Agreements

Earnings

Payouts

Audit Logs

Commercial Agreements

Active agreement versions across all influencers. Manage versions from each influencer's profile.

Influencer	Version	Model	Terms	Scope	Effective
Amina Khan	v1	FIXED PER SALE	₹750	All courses	01 Aug 2026 → onwards
Priya Verma	v1	PERCENTAGE COMMISSION	15%	All courses	01 Jul 2026 → onwards
Rahul Sharma	v2	REVENUE SHARE	50% / 50%	AI Mastery	01 Jun 2026 → onwards
Rahul Sharma	v1	REVENUE SHARE	40% / 60%	All courses	01 Jun 2026 → onwards
Amina Khan	v1	FIXED PER SALE	₹750	All courses	01 Aug 2026 → onwards
Priya Verma	v1	PERCENTAGE COMMISSION	15%	All courses	01 Jul 2026 → onwards
Rahul Sharma	v2	REVENUE SHARE	50% / 50%	AI Mastery	01 Jun 2026 → onwards
Rahul Sharma	v1	REVENUE SHARE	40% / 60%	All courses	01 Jun 2026 → onwards

Super Admin

Sign out

Influencer Partner Platform

Dashboard

Influencers

Courses

Campaigns

Tracking Links

Coupons

Sales

Commercial Agreements

Earnings

Payouts

Audit Logs

Sales

10 orders

Export CSV

Record Sale

START DATE

mm/dd/yyyy

END DATE

mm/dd/yyyy

COURSE

All courses

CAMPAIGN

All campaigns

COUPON CODE

RAHUL10

STATUS

All statuses

UTM SOURCE

youtube

UTM MEDIUM

influencer

UTM CAMPAIGN

dedicated

UTM CONTENT

sf

UTM TERM

harshpriyam

Apply filters

ORDER	DATE	COURSE	INFLUENCER	COUPON	UTM SOURCE / MEDIUM	CAMPAIGN / CONTENT / TERM	FINAL AMOUNT	EARNINGS	STATUS
ORD-MUDJS9YP-WC26	18 Sept 2026, 12:00 am	Python + DSA Mastery	Rahul Sharma	RAHUL10	—	—	₹10,000	₹4,000	SUCCESSFUL
ORD-MUCV5JBY-1SPU	18 Sept 2026, 12:00 am	Python + DSA Mastery	Rahul Sharma	RAHUL10	—	—	₹10,000	₹4,000	SUCCESSFUL
ORD-MUDJS9Y4-3YDV	15 Sept 2026, 12:00 am	Data Science Bootcamp	Amina Khan	AMINA750	—	—	₹11,000	₹750	PARTIALLY REFUNDED
ORD-MUCV5JBG-IFBP	15 Sept 2026, 12:00 am	Data Science Bootcamp	Amina Khan	AMINA750	—	—	₹11,000	₹750	PARTIALLY REFUNDED
ORD-MUDJS9XM-RGEH	12 Sept 2026, 12:00 am	AI Mastery	Priya Verma	PRIYA15	—	—	₹12,750	₹1,913	SUCCESSFUL
ORD-MUCV5JAW-BTJQ	12 Sept 2026, 12:00 am	AI Mastery	Priya Verma	PRIYA15	—	—	₹12,750	₹1,913	SUCCESSFUL
ORD-MUDJS9WZ-VES1	10 Sept 2026, 12:00 am	AI Mastery	Rahul Sharma	—	youtube / influencer	september_launch / rahul_video_1 / —	₹15,000	₹7,500	SUCCESSFUL

No screenshots still need to be captured manually — all listed pages were reachable and captured directly from the live dev instance during this audit.

20. User Flows

Login

Influencer/Admin → /login → Auth.js Credentials check → role-based redirect (/influencer or /admin)

Sales Tracking

Influencer → Sales Tracking nav item → select date range (+ optional filters) → Apply Filters
→ GET /api/influencer/sales-tracking → session-derived identity → adapter → results rendered

Order → Earnings → Payout

Admin records a sale (manual entry, only path implemented today)
→ attribution resolved (coupon takes priority over tracking link)
→ applicable CommercialAgreementVersion resolved (coupon-linked > campaign-scoped > course-scoped > general)
→ calculateSplit() computes eligible revenue + influencer/company split (decimal.js)
→ FinancialTransaction (EARNING, PENDING) created with a full immutable snapshot of the terms used
→ Finance/Admin approves the transaction (PENDING → APPROVED)
→ Payout created, bundling approved transactions → marked PAID once money moves (external to the app)

Refund

Admin records a refund against an order
→ proportional reversal calculated against the original EARNING transaction
→ a REVERSAL transaction is created, linked back to the original via relatedTransactionId

21. Current Status

Feature	Status	Production Ready?	Tech Team Action Required?
Auth / RBAC / row-level scoping	Implemented	Yes (pending secret rotation)	Rotate secrets; confirm session expiry policy
Influencer/course/campaign/coupon/tracking-link management	Implemented	Yes	No
Commercial agreements & financial calculation engine	Implemented	Yes	No

Feature	Status	Production Ready?	Tech Team Action Required?
Payouts	Implemented (record-keeping only)	Yes, if manual money movement is acceptable	Confirm whether a real payment-provider integration is needed
Sales Tracking	Mock data	No	Implement real <code>SalesTrackingAdapter</code> (\$10)
Order ingestion	Manual entry only	Partially	Decide/build real ingestion path if manual entry isn't sufficient at scale
Marketing assets	Partially implemented	No file upload	Add upload capability if needed
CSV exports	Implemented	Yes at current scale (10k row cap)	Revisit if data volume grows significantly
Testing	One manual smoke script	No	Requires integration/CI test coverage before launch
Deployment/CI	Not implemented	No	Requires integration
Monitoring	Not implemented	No	Requires integration

22. Known Limitations

- Sales Tracking runs entirely on synthetic mock data — no real database table or Metabase connection exists yet.
- Order ingestion is manual-entry only; API/webhook/CSV-import paths are reserved in the schema (`OrderSource` enum) but not built.
- No payment gateway integration — payouts are recorded, not actually disbursed by the app.
- No email/WhatsApp/SMS notification delivery — notifications are in-app only.
- No file upload capability — marketing asset URLs must be supplied externally.
- No automated unit/integration test suite — one manual, DB-dependent Playwright smoke script only.
- No CI/CD pipeline.
- No monitoring/error-tracking integration (e.g. Sentry) — the only server-side logging is a single `console.error` in the shared API error handler.
- No password-reset flow, SSO, or MFA.
- `next-auth` is on a pre-1.0 beta release.
- `legacy-portfolio/` at the repo root is unrelated leftover content from this repo's origin as a personal GitHub Pages site.

23. Testing

Existing tests: none in the traditional sense — no Jest/Vitest/Mocha, no `*.test.ts / *.spec.ts` files, no `jest.config.* / playwright.config.*`.

What does exist: `scripts/e2e-smoke.mjs`, run via `npm run e2e:smoke`. A plain Node script (not the Playwright test runner) that launches Chromium directly, logs in as the seeded influencer (`rahul@creator.dev`), and walks through RBAC enforcement, the full order → attribution → calculation → refund → approval → payout pipeline through the real UI, and the 4 CSV export endpoints. Requires a running dev server and a seeded database; not wired into any CI (none exists).

What's not covered: no coverage of the Sales Tracking feature specifically, no admin-side UI flows beyond what the smoke script touches, no unit-level coverage of the financial calculation engine's edge cases (e.g. HYBRID model thresholds, multi-partial-refund sequences).

Recommended pre-launch smoke-test checklist

```
[ ] Login (each role) / Logout
[ ] Role-based redirect and route gating (middleware)
[ ] Influencer cannot access another influencer's data via any UI action or direct API call
[ ] Sales Tracking: date filter, each UTM filter individually, coupon filter, course filter, combined filters
[ ] Sales Tracking: empty-result state, error state (simulate a backend failure)
[ ] Admin: create/edit an agreement, record a sale, approve earnings, create a payout, process a refund
[ ] CSV exports (all 4) produce correct, complete data
[ ] Mobile UI spot-check (drawer nav, filter grid reflow)
[ ] Security: attempt to pass another influencer's id/email as a parameter anywhere it's accepted client-side
```

24. Monitoring & Operations

Not currently implemented. No uptime monitoring, no APM, no error-tracking SDK, no structured logging — the only current error visibility is a single `console.error(err)` call inside the shared API error handler (`src/lib/api/handler.ts`), which will only reach whatever the hosting platform captures from `stdout/stderr`.

Recommended for the tech team to add before/at launch: application uptime monitoring, API error-rate tracking, database error/connection monitoring, auth failure tracking, and (once a real Sales Tracking adapter exists) connectivity/response-time monitoring for that data source.

25. Backup & Rollback

Not currently defined anywhere in the codebase — REQUIRES TECH TEAM CONFIRMATION for: - Database backup frequency/retention strategy. - Application deployment rollback approach (no CI/CD exists yet to define this against). - Database migration rollback plan (Prisma migrations are forward-only by default; a rollback plan needs to be decided by the team operating the production database).

26. Handover Summary

What is already built

A complete, working influencer partnership platform: auth + two-layer RBAC/data-isolation, full commercial-agreement versioning and a decimal-precise financial calculation engine, attribution, tracking links, coupons, campaigns, courses, payouts, refunds/reversals, goals, notifications, audit logging, CSV exports, and a fully-built Sales Tracking UI/API against a clean swappable adapter interface.

What the tech team needs to connect

A real production Postgres database; a real Sales Tracking data source (Metabase or otherwise) behind the existing `SalesTrackingAdapter` interface; a real order-ingestion path if manual entry isn't sufficient; production secrets and a hosting/CI/CD pipeline (none exist today).

What needs to be tested

Full RBAC/data-isolation re-verification against production data; the pre-launch smoke-test checklist (§23); a dedicated security pass (§14 Critical list).

What needs to be configured before launch

Environment variables and secrets (§8); hosting, domain, SSL (§15); database migrations-on-deploy; monitoring and backup strategy (§24–25).

Recommended for Phase 2

Real payment/LMS order ingestion (API or webhook); file upload for marketing assets; email/WhatsApp notification delivery; automated test suite + CI; password reset/SSO; pagination and export for the Sales Tracking detail table at scale.